Building a Programming Language from Scratch

Capstone 3.0

Your Language

Your Rules

Teach the Gremlin

*SOME ASSEMBLY REQUIRED

Ten Instruction Pascal Subset

What is TIPS?

BORING!

- Ten core instructions
- Fully defined by formal grammar
- Deterministic and unambiguous
- Small enough to implement
- Powerful enough to compute

What WE are defining as our 10 core instructions necessary to fully implement any program our language accepts.

“The” Ten Instructions of TIPS

- **PROGRAM** – define the program entry point
- **VAR** – declare variables
- **BEGIN / END** – define a compound block
- **ASSIGN (:=)** – update variable state
- **READ** – accept input
- **WRITE** – produce output
- **IF** – conditional branching
- **WHILE** – repetition
- **Arithmetic Expressions** – compute values
- **Relational Comparison** – evaluate truth

10 Just Looks Good

- The textbook had its “core 10.”
  We have ours.

- Ten isn’t sacred – it’s just tidy.

- The reality is that a programming language needs surprisingly little to be powerful.

  Once you have state, branching, and repetition, you can compute anything.

- Turing completeness isn’t counting how many bullets are on your slide.

Building Something New

HOW UNIQUE IS COMPLETELY UP TO YOU

TIPS gives us rules

TIPSY gives us freedom

What you build this semester has never existed before.

Ten Instruction Pascal Subset - YOUR WAY

What is TIPS-Y?

TIPSY is what happens when a formal language becomes yours. The grammar defines the structure – but you decided how parts of it look, feel, and behave.

Your class customizations:

- SPAWN as a custom statement
- Block Comments (in addition to line comments)
- Better Error Messages
- Compound Assignment Statements

Same architecture. Same rules. Different personality.

The Entire Language

Where we will end up...

That's A Lot of Grammars!

program → PROGRAM IDENT SEMICOLON block

block → [VAR IDENT COLON (REAL | INTEGER) SEMICOLON
{ IDENT COLON (REAL | INTEGER) SEMICOLON }]
compound

Or if you prefer to look at it as a one liner:
[VAR IDENT COLON (REAL | INTEGER) SEMICOLON { IDENT COLON (REAL | INTEGER) SEMICOLON }] compound

statement → assign | compound | read | write | if | while | CUSTOM

if → IF expression THEN statement [ELSE statement]

while → WHILE expression statement

assign → IDENT (ASSIGN | CUSTOM_OPER) value

compound → TOK_BEGIN statement { SEMICOLON statement } END

read → READ OPENPAREN IDENT CLOSEPAREN

write → WRITE OPENPAREN (IDENT | STRINGLIT) CLOSEPAREN

expression → value [relOp value]

relOp → LESSTHAN | GREATERTHAN | EQUALTO | NOTEQUALTO

value → term { (PLUS | MINUS | OR) term }

term → factor { (MULTIPLY | DIVIDE | MOD | AND) factor}

factor → [MINUS | NOT] primary

primary → FLOATLIT | INTLIT | IDENT | OPENPAREN value CLOSEPAREN

DONT WORRY!

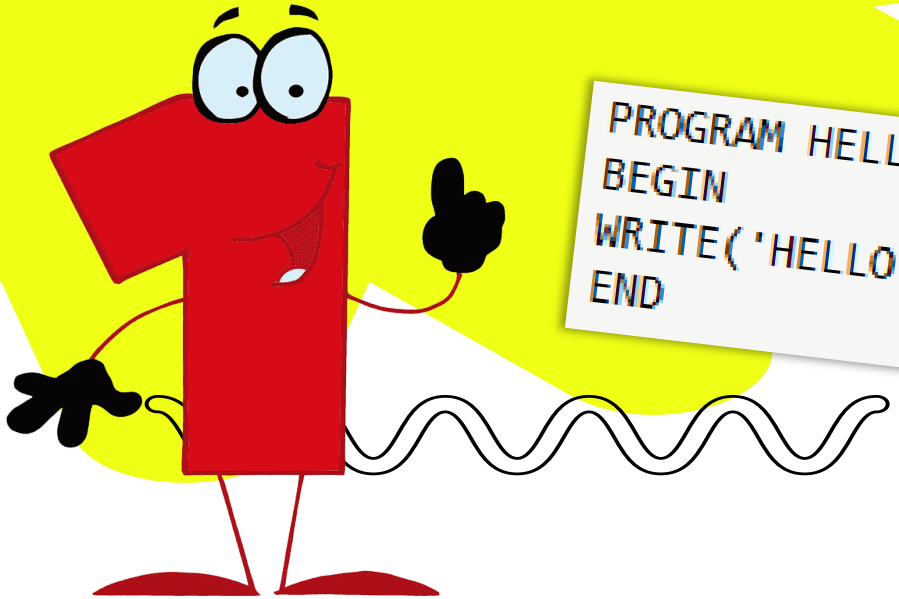
We will Divide The Work into Four Parts

8+ weeks To Cross The Finish Line

Hello World!

```
program → PROGRAM  IDENT  SEMICOLON  block

block → compound

statement → compound | write

compound → TOK_BEGIN  statement  {SEMICOLON  statement}  END

write → WRITE  OPENPAREN  STRINGLIT  CLOSEPAREN
```

Languages aren't built all at once. They're grown.



```
PROGRAM HELLO;
BEGIN
WRITE('HELLO WORLD!')
END
```

- Build a complete, runnable language core
- Support structured block nesting
- Implement a functioning output statement
- Validate syntax with recursive descent
- Establish the foundation for the full grammar

You'll be able to...

```
PROGRAM CHAOS5;
BEGIN
  WRITE('Capstone loading...');
  BEGIN
    WRITE('Parsing tokens...');
    BEGIN
      WRITE('Building AST...')
    END;
    WRITE('Interpreting...')
  END;
  WRITE('And somehow it worked.')
END
```

```
PROGRAM CHAOS2;
BEGIN
  WRITE('Outer block reporting for duty.');
  BEGIN
    WRITE('Inner block whispering...');
    WRITE('We could have stayed simple.')
  END;
  WRITE('But no. We chose nesting.')
END
```

```
PROGRAM        GREMLIN    ;

BEGIN

        WRITE      (      'Spacing is a social construct.'      )      ;

   BEGIN

WRITE(   'Tabs?   Spaces?      Who knows.'   )

   END     ;

        WRITE   (   'If this fails, your lexer is judging whitespace.'   )

END
```

PART TWO

Giving TIPSY a Memory

```
program → PROGRAM IDENT SEMICOLON block

block → [ VAR IDENT COLON ( REAL | INTEGER ) SEMICOLON
          { IDENT COLON ( REAL | INTEGER ) SEMICOLON } ]
          compound

statement → assign | compound | read | write

assign → IDENT ( ASSIGN | CUSTOM_OPER ) value

compound → TOK_BEGIN statement { SEMICOLON statement } END

read → READ OPENPAREN IDENT CLOSEPAREN

write → WRITE OPENPAREN ( IDENT | STRINGLIT ) CLOSEPAREN

primary → FLOATLIT | INTLIT | IDENT
```

It remembers..it learns..it assigns.

- Parse and store declarations
- Enforce INTEGER vs REAL typing
- Design the symbol table structure
- Support input into variables
- Enable assignment and state changes

This is where it gets heavy

You'll be able to...

```
PROGRAM OFFERING;
VAR
  X : INTEGER;
BEGIN
  WRITE('THE MACHINE DEMANDS A NUMBER.');
  READ(X);
  WRITE('YOUR NUMBER HAS BEEN ACCEPTED.');
  WRITE('THE GREMLIN IS PLEASED.')
END
```

```
PROGRAM COPY;
VAR
  A : INTEGER;
  B : INTEGER;
BEGIN
  WRITE('ENTER A NUMBER:');
  READ(A);
  B := A;
  WRITE('A WAS:');
  WRITE(A);
  WRITE('B IS:');
  WRITE(B)
END
```

```
PROGRAM   BOSS1;

VAR
 A1:INTEGER;
B2 : INTEGER;
 TEMP2:REAL;

BEGIN

WRITE('WELCOME TO THE INPUT CAVE.');
 WRITE ( 'ENTER A1:' ) ;
READ( A1 );

WRITE('ENTER B2:');READ(B2);

BEGIN
 B2:=A1;
 WRITE('B2 NOW IS:');
 WRITE(  B2  )
END;

WRITE('ENTER TEMP2 (REAL):');
   READ( TEMP2 );

WRITE('TEMP2 STORED AS:');
WRITE(TEMP2);

WRITE('END. NO MATH WAS HARMED.')
END
```

PART THREE

TIPSY Learns Math and Manners

```
program → PROGRAM IDENT SEMICOLON block

block → [ VAR IDENT COLON ( REAL | INTEGER ) SEMICOLON
          { IDENT COLON ( REAL | INTEGER ) SEMICOLON } ]
          compound

statement → assign | compound | read | write

assign → IDENT ( ASSIGN | CUSTOM_OPER ) value

compound → TOK_BEGIN statement { SEMICOLON statement } END

read → READ OPENPAREN IDENT CLOSEPAREN

write → WRITE OPENPAREN ( IDENT | STRINGLIT ) CLOSEPAREN

value → term { ( PLUS | MINUS ) term }

term → factor { ( MULTIPLY | DIVIDE | MOD ) factor }

factor → { MINUS } primary

primary → FLOATLIT | INTLIT | IDENT | OPENPAREN value CLOSEPAREN
```

ALMOST DONE!!!

Not everyone gets to go first.

- Implement arithmetic operators (+ - * / MOD)
- Enforce operator precedence
- Support unary operations
- Support binary operations
- Evaluate nested expressions

You'll be able to...

```
PROGRAM PRECTEST;

VAR
  RES : INTEGER;
  R : REAL;

BEGIN

WRITE('PRECEDENCE TEST: NO PARENTHESES. ONLY VIBES.');
WRITE('IF YOUR ANSWERS ARE WRONG, YOUR AST IS DOING STREET MAGIC.');

WRITE('INTEGER TRICK (EXPECTED RES = 104):');
RES := 100 / 4 * 3 + 7 / 2 * 4 + 5 * 6 / 3 - 8 + 9 * 2 - 10 / 3;
WRITE('ACTUAL RES =');
WRITE(RES);

WRITE('REAL TRICK (EXPECTED R = 2.2):');
R := 1.0 + 2.0 * 3.0 - 4.0 / 5.0 + 6.0 / 4.0 * 2.0 - 7.0;
WRITE('ACTUAL R =');
WRITE(R);

WRITE('REMEMBER: * AND / BEFORE + AND -, AND LEFT TO RIGHT WITHIN SAME LEVEL.');
WRITE('END OF SHOW. TIP YOUR LEXER.')

END
```

```
PROGRAM MAGIC5;

VAR
  NUM : INTEGER;  ORIG : INTEGER;  REALX : REAL;  RESULT : REAL;

BEGIN

WRITE('MATH MAGIC TRICK.');
WRITE('PICK ANY INTEGER AND ENTER IT.');

READ(NUM);
ORIG := NUM;
WRITE('I WILL MULTIPLY IT BY 2.');
NUM := NUM * 2;
WRITE('I WILL ADD 10.');
NUM := NUM + 10;
WRITE('I WILL DIVIDE BY 2.');
NUM := NUM / 2;
WRITE('I WILL SUBTRACT YOUR ORIGINAL NUMBER.');
NUM := NUM - ORIG;
WRITE('THE RESULT SHOULD NOW BE 5.');
WRITE('ACTUAL RESULT:');
WRITE(NUM);

WRITE('--------------------------------');

WRITE('NOW A REAL NUMBER TRICK.');
WRITE('ENTER ANY REAL NUMBER.');
READ(REALX);
WRITE('I COMPUTE: REALX * 3.0 - REALX / 2.0 + 1.0');
RESULT := REALX * 3.0 - REALX / 2.0 + 1.0;
WRITE('THIS TESTS OPERATOR PRECEDENCE.');
WRITE('ACTUAL RESULT:');
WRITE(RESULT);
WRITE('IF 5 WAS NOT 5, CHECK YOUR INTEGER DIVISION.');
WRITE('IF THE REAL RESULT LOOKS WRONG, CHECK PRECEDENCE.');
WRITE('END OF SHOW.')

END
```

PART FOUR

TIPSY Learns Truth and Consequences

What is Truth?

```
program → PROGRAM IDENT SEMICOLON block

block → [ VAR IDENT COLON ( REAL | INTEGER ) SEMICOLON
          { IDENT COLON ( REAL | INTEGER ) SEMICOLON } ]
          compound

Or if you prefer to look at it as a one liner:
[ VAR IDENT COLON ( REAL | INTEGER ) SEMICOLON { IDENT COLON ( REAL | INTEGER ) SEMICOLON } ] compound

statement → assign | compound | read | write | if | while | CUSTOM

if → IF expression THEN statement [ ELSE statement ]

while → WHILE expression statement

assign → IDENT ( ASSIGN | CUSTOM_OPER ) value

compound → TOK_BEGIN statement { SEMICOLON statement } END

read → READ OPENPAREN IDENT CLOSEPAREN

write → WRITE OPENPAREN ( IDENT | STRINGLIT ) CLOSEPAREN

expression → value [ relOp value ]

relOp → LESSTHAN | GREATERTHAN | EQUALTO | NOTEQUALTO

value → term { ( PLUS | MINUS | OR ) term }

term → factor { ( MULTIPLY | DIVIDE | MOD | AND ) factor }

factor → [ MINUS | NOT ] primary

primary → FLOATLIT | INTLIT | IDENT | OPENPAREN value CLOSEPAREN
```

- Introduce comparison and truth evaluation
- Implement branching and repetition
- Direct execution through logical control flow
- Finalize all semester customizations
- Assemble the complete TIPS language

AND A FUN OPPORTUNITY FOR BONUS POINTS!

You'll be able to...

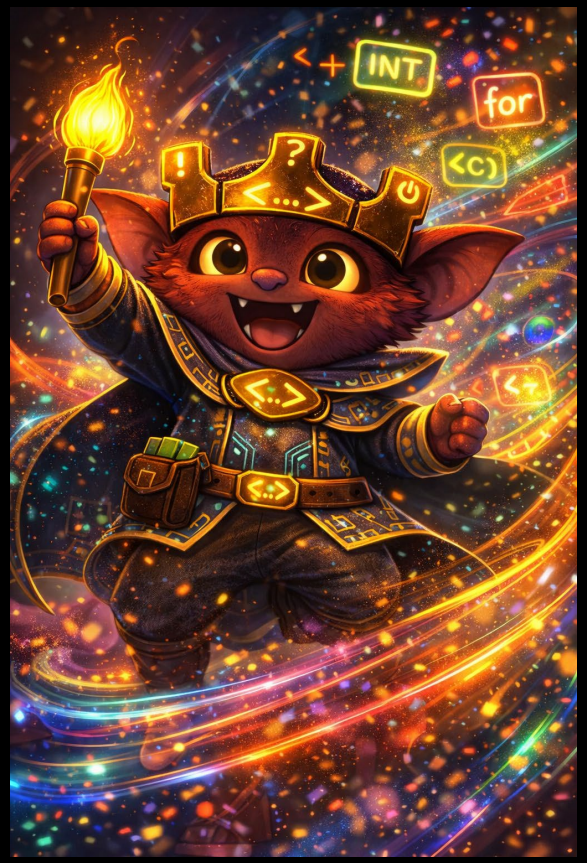
CODE ANYTHING!

YOUR LANGUAGE IS NOW TURING COMPLETE

Seriously...
it only takes a
couple of minutes

Easy 10 Points on your Lowest Test Score

Bonus Points



Implement “skins” for your language

You'll be able to...

```
VIBE ECHOTST;

FLEX
  X : INTEGER;
GETLIT
        SPAWN;
  ECHO('DROP A NUMBER:');
  SNATCH(X);
  ECHO('YOU SAID:');
  ECHO(X)
YEET
```

```
meow X meoww

MEOw
  X meOW MEOOow meoww

mEoW
  MeOw meowwwW 'meow me a number' meowwwWW meoww
  meoW meowwwW X meowwwWW meoww
  MeOw meowwwW X meowwwWW
MEOW
```

Your creativity gets to shine here!

OK…
but how the hell do we get started?

We build it in layers:

text → tokens → structure → meaning.

Rule It. Shape It. Parse It. Run It.

Get It In Your Head

It's the Same Song and Dance Every Time

1. Define the Rules
- Add new rules

2.Build the Structures
- Give your nodes their right stuff

3.Parse the Patterns
- Add new parsers to assemble your nodes

4.Make It Do the Thing
- Add new interprets to your structures

SOME EXCEPTIONS MAY OCCUR

There Will Be Exceptions

- The four-step pattern still applies
- But some features require deeper understanding
- You may need to modify supporting files
- Not everything lives in rules.l, parser.cpp, or ast.h
- Sometimes you must understand the whole system

And sometimes even change it!

This is not one file.
It is a machine.

The Skeleton You've Been Given

Core Files You Will Live In

- rules.l
  → Defines tokens (lexer rules)
- parser.cpp
  → Recursive descent parser
- ast.h
  → AST node structures + interpret logic

The Infrastructure

- **driver.cpp**
  → Program entry point + argument handling
- **lexer.h**
  → Token definitions and interfaces
- **debug.h**
  → Debug utilities and tracing support
- **Makefile**
  → Controls how everything builds together

Will You Rise to the Challenge?

What This Is Really Testing

It's not a bug if it's a **FEATURE**

Can you extend a non-trivial codebase?

Can you map formal grammar to working code?

Can you design clean data structures?

Can you reason recursively?

Can you debug something you didn't write?

Can you build in layers without breaking previous work?

Can you think like a designer?

Before You Get Started

Panic is not a development strategy.

Build and test in small pieces

Do not wait until the last minute

Ask for help early and often

There is no single "correct" implementation

Many different designs can be successful

Outputs may differ across implementations

My solution was ~1500 lines of code

Not everyone finishes

I give about half to you

This project is 40% of your final grade

You CAN DO This

COME TO LABS FOR HELP AND MORE DETAILS

Good Luck!

This message is CossittBot Approved

"Built a programming language from scratch" belongs on your résumé.

This is not a small project. It's a milestone.